EVALUACIÓN 2 GUIADA

Databricks Data Engineering & Lakehouse sobre AWS

Sesiones 07 a 11

Cliente	La Positiva
Fecha	Miércoles 28 de octubre de 2026
Modalidad	Evaluación guiada / práctica
Duración técnica	90 minutos
Stack	Databricks + Amazon S3 + Bitbucket + Delta Lake + Lakeflow

Versión para revisión del instructor

El bucket/prefijo S3 real y la URL de Bitbucket se mantienen como placeholders hasta su confirmación.

1 Propósito y alcance de la evaluación

Caso práctico guiado: desde almacenamiento S3 hasta despliegue versionado.

1.1 Objetivo

Construir y desplegar de forma guiada una solución de Data Engineering para el dominio de seguros utilizando los componentes trabajados en las sesiones 07 a 11.

- Procesamiento PySpark: Window Functions, deduplicación, limpieza y agregaciones.
- Delta Lake: MERGE/UPSERT, schema evolution e idempotencia.
- Lakeflow: Declarative Pipeline y Job con dependencias.
- Ingeniería de software: metadata, parametrización, estructura de repositorio y Git.
- Versionamiento y despliegue: Bitbucket + Databricks Git Folder + Declarative Automation Bundles.

1.2 Reglas

- La evaluación es individual y guiada por el instructor.
- No se permite utilizar herramientas externas de Inteligencia Artificial durante la ejecución.
- No se deben registrar secretos, tokens, Access Keys o contraseñas dentro del repositorio.
- Se evaluará la ejecución y las evidencias; no solamente el código final.

1.3 Material entregado

Archivo / paquete	Uso
evaluacion2-datasets-volume.zip	3 CSV para cargar al Volume externo.
evaluacion2-seguros-databricks_REPO.zip	Código PySpark, SQL y definiciones YAML de Job/Pipeline.
Este documento	Guía completa paso a paso.

2 Arquitectura objetivo

El volumen lapositiva gobierna archivos en S3; Bitbucket conserva código y definiciones de despliegue.



Figura 1. Creación conceptual del volumen lapositiva. La URI S3 de la lámina es referencial.

3 Crear el nuevo Volume externo lapositiva

Unity Catalog gobierna la ruta S3; el nombre lógico será workspace.bronze.lapositiva.

3.1 Prerrequisitos administrativos

- El bucket/prefijo S3 debe estar incluido dentro de una External Location de Unity Catalog.
- El usuario necesita USE CATALOG sobre workspace y USE SCHEMA + CREATE VOLUME sobre bronze.
- Para un External Volume necesita además CREATE EXTERNAL VOLUME sobre la External Location que cubre la ruta S3.

Recomendación. Crear el volumen sobre un subdirectorio dedicado de la External Location, no sobre la raíz compartida.

3.2 Crear catálogo/esquema y Volume

Ejecutar en Databricks SQL Editor

```
USE CATALOG workspace;

CREATE SCHEMA IF NOT EXISTS bronze;

CREATE EXTERNAL VOLUME IF NOT EXISTS workspace.bronze.lapositiva
LOCATION '<S3_URI_LAPOSITIVA>'
COMMENT 'Volume externo para laboratorios La Positiva sobre Amazon S3';

DESCRIBE VOLUME workspace.bronze.lapositiva;
```

3.3 Ruta lógica resultante

```
/Volumes/workspace/bronze/lapositiva/
```

3.4 Crear las carpetas de la evaluación

Notebook Python / PySpark

```
VOLUME = "/Volumes/workspace/bronze/lapositiva"

for path in [
  f"{VOLUME}/evaluacion2/siniestros",
  f"{VOLUME}/evaluacion2/pagos",
]:
  dbutils.fs.mkdirs(path)

for item in dbutils.fs.ls(f"{VOLUME}/evaluacion2"):
  print(item.path)
```

3.5 Resultado esperado

```
/Volumes/workspace/bronze/lapositiva/evaluacion2/
├── siniestros/
│   ├── siniestros_eval2_base.csv
│   └── siniestros_eval2_incremental.csv
└── pagos/
    └── pagos_eval2.csv
```

4 Crear el repositorio Bitbucket y conectarlo a Databricks

Bitbucket será la fuente de verdad del código y de las definiciones del Job/Pipeline.

4.1 Crear repositorio

1. En Bitbucket Cloud, crear un repositorio privado llamado evaluacion2-seguros-databricks.
2. Mantener main como rama principal.
3. Copiar la URL HTTPS del repositorio: <BITBUCKET_REPO_URL>.

4.2 Conectar Bitbucket con Databricks

4. Generar un API token de Bitbucket con permisos de lectura y escritura del repositorio.
5. En Databricks: Settings > Linked accounts > Add Git credential.
6. Seleccionar Bitbucket e ingresar la credencial/token.

Permisos recomendados para usuario. Bitbucket Cloud: read:repository:bitbucket y write:repository:bitbucket.

4.3 Crear Git Folder

7. Databricks > Workspace > Create > Git folder.
8. Repository URL: <BITBUCKET_REPO_URL>.
9. Provider: Bitbucket.
10. Git folder name: evaluacion2-seguros-databricks.
11. Crear o cambiar a la rama feature/evaluacion2.

4.4 Flujo Git de la evaluación

```
git status
git switch -c feature/evaluacion2

git add .
git commit -m "Evaluacion 2 guiada - pipeline seguros"
git push -u origin feature/evaluacion2
```

Alternativa desde la interfaz. En un Git Folder también se puede usar el panel Git de Databricks para branch, commit, push y pull. El ejercicio mantiene los comandos porque forman parte de la sesión 11.

5 Estructura del proyecto

Código versionado en Bitbucket; datos excluidos mediante .gitignore.

```
evaluacion2-seguros-databricks/
├── databricks.yml
├── resources/
│   ├── eval2_job.job.yml
│   └── eval2_pipeline.pipeline.yml
├── src/
│   ├── 01_bronze_eval2.py
│   ├── 02_silver_eval2.py
│   ├── 03_gold_pipeline_eval2.sql
│   └── 04_validacion_eval2.py
├── sql/
│   └── 05_validacion_sql.sql
├── docs/
│   └── EVIDENCIAS.md
├── .gitignore
└── README.md
```

5.1 Qué se versiona y qué no

Elemento	Ubicación	¿Bitbucket?
CSV de siniestros y pagos	External Volume / Amazon S3	No
Notebooks / scripts PySpark	src/	Sí
SQL del Pipeline	src/ y sql/	Sí
Job y Pipeline	resources/*.yml	Sí
Parámetros y targets	databricks.yml	Sí
Evidencias/capturas	docs/ o entrega separada	Opcional

5.2 .gitignore

```
# Datos: viven en el Volume/S3, no en Git
*.csv
data/
datasets/

# Python
__pycache__/
*.pyc
.venv/

# Databricks / tooling local
.databricks/
dist/
```

6 Cargar los datasets al Volume

Los archivos se cargan en lapositiva/evaluacion2; no se copian al repositorio.

6.1 Archivos

Archivo	Ruta destino
siniestros_eval2_base.csv	/Volumes/workspace/bronze/lapositiva/evaluacion2/siniestros/
siniestros_eval2_incremental.csv	/Volumes/workspace/bronze/lapositiva/evaluacion2/siniestros/
pagos_eval2.csv	/Volumes/workspace/bronze/lapositiva/evaluacion2/pagos/

6.2 Procedimiento

12. Catalog Explorer > workspace > bronze > Volumes > lapositiva.
13. Ingresar a evaluacion2/siniestros y cargar los dos archivos de siniestros.
14. Ingresar a evaluacion2/pagos y cargar pagos_eval2.csv.
15. Validar desde un notebook con dbutils.fs.ls().

```
BASE = "/Volumes/workspace/bronze/lapositiva/evaluacion2"

display(dbutils.fs.ls(f"{BASE}/siniestros"))
display(dbutils.fs.ls(f"{BASE}/pagos"))
```

7 Paso práctico 1: Bronze

Leer CSV desde el Volume y crear tablas Delta con metadata técnica.

7.1 Parámetros del notebook

```
dbutils.widgets.text("catalog", "workspace")
dbutils.widgets.text(
  "volume_base",
  "/Volumes/workspace/bronze/lapositiva/evaluacion2"
)
```

7.2 Tablas esperadas

- workspace.bronze.eval2_siniestros_base
- workspace.bronze.eval2_siniestros_incremental
- workspace.bronze.eval2_pagos

7.3 Metadata técnica

Columna	Propósito
fecha_ingestion	Fecha/hora en que el archivo se carga a Bronze.
archivo_origen	Nombre del archivo fuente.
id_ejecucion	Identificador único de la ejecución.

Código del ejercicio. El script completo 01_bronze_eval2.py se incluye en el Anexo A y en el repositorio entregado.

7.4 Resultado de control

Tabla	Conteo esperado
eval2_siniestros_base	48
eval2_siniestros_incremental	13
eval2_pagos	60

8

Paso práctico 2: Silver + MERGE

Última versión por clave, limpieza de pagos, schema evolution e idempotencia.

8.1 Deduplicación con Window

```
w = (
   Window
   .partitionBy("id_siniestro")
   .orderBy(F.col("fecha_actualizacion").desc())
)

base_latest = (
   spark.table(BASE)
   .withColumn("rn", F.row_number().over(w))
   .filter(F.col("rn") == 1)
   .drop("rn")
)
```

8.2 MERGE incremental

```
(
   target.alias("target")
   .merge(
      inc_latest.alias("source"),
      "target.id_siniestro = source.id_siniestro"
   )
   .withSchemaEvolution()
   .whenMatchedUpdateAll(
      condition=(
         "source.fecha_actualizacion > "
         "target.fecha_actualizacion"
      )
   )
   .whenNotMatchedInsertAll()
   .execute()
)
```

8.3 Idempotencia

- Ejecutar el mismo MERGE por segunda vez.
- El conteo debe mantenerse en 40 siniestros.
- La nueva columna prioridad debe quedar incorporada en la tabla Silver.

8.4 Limpieza de pagos

- Eliminar duplicados por id_pago.
- monto_pago nulo → 0.
- medio_pago vacío o nulo → NO_INFORMADO.

8.5 Resultados de control

Objeto	Resultado esperado
workspace.silver.eval2_siniestros	40 IDs únicos
Duplicados de id_siniestro	0
workspace.silver.eval2_pagos	55 pagos únicos

9 Paso práctico 3: Gold Declarative Pipeline

Materialized Views y DAG de datasets.

9.1 Pipeline

- Nombre: PIPELINE_EVAL2_GOLD
- Catalog: workspace
- Schema: gold

9.2 DAG de datasets

```
workspace.silver.eval2_pagos
        ↓
eval2_pagos_por_siniestro
        ↓
eval2_resumen_siniestros
        ↑
workspace.silver.eval2_siniestros
```

9.3 Materialized Views

- eval2_pagos_por_siniestro: cantidad de pagos, pagos procesados y monto pagado procesado.
- eval2_resumen_siniestros: resumen por tipo de siniestro con cantidad, reserva, monto pagado y pagos procesados.

Código del Pipeline. El archivo 03_gold_pipeline_eval2.sql está versionado en src/ y se referencia desde eval2_pipeline.pipeline.yml.

10

Guardar el Job en Bitbucket: Job as Code

La definición del Job se versiona en resources/eval2_job.job.yml y se despliega con el Bundle.

10.1 ¿Por qué no basta crear el Job manualmente?

Un Job creado o editado solamente en la UI existe en el workspace, pero su configuración no se escribe automáticamente en Bitbucket. Para que el repositorio sea la fuente de verdad, la definición debe estar expresada como YAML dentro del Bundle.

Patrón recomendado. Código y definiciones YAML en Bitbucket → Bundle validate/deploy → Databricks crea o actualiza Job y Pipeline.

10.2 DAG del Job

```
01_bronze
  ↓
02_silver_merge   (max_retries = 1)
  ↓
03_gold_pipeline
  ↓
04_validacion
```

10.3 Archivo resources/eval2_job.job.yml

```yaml
resources:
  jobs:
    eval2_job:
      name: JOB_EVAL2_SEGUROS

      queue:
        enabled: true

      parameters:
        - name: catalog
          default: ${var.catalog}

        - name: volume_base
          default: ${var.volume_base}

      tasks:
        - task_key: 01_bronze
          notebook_task:
            notebook_path: ../src/01_bronze_eval2.py

        - task_key: 02_silver_merge
          depends_on:
            - task_key: 01_bronze
          max_retries: 1
          min_retry_interval_millis: 60000
          notebook_task:
            notebook_path: ../src/02_silver_eval2.py

        - task_key: 03_gold_pipeline
          depends_on:
            - task_key: 02_silver_merge
          pipeline_task:
            pipeline_id: ${resources.pipelines.eval2_gold.id}
            full_refresh: false

        - task_key: 04_validacion
          depends_on:
            - task_key: 03_gold_pipeline
          notebook_task:
            notebook_path: ../src/04_validacion_eval2.py
```

11 Desplegar Job y Pipeline desde el repositorio

Declarative Automation Bundles convierte los YAML versionados en recursos Databricks.

11.1 databricks.yml

```
bundle:
  name: evaluacion2-seguros-databricks

include:
  - resources/*.yml

variables:
  catalog:
    description: Catalog de la evaluación
    default: workspace

  volume_base:
    description: Ruta base del Volume
    default: /Volumes/workspace/bronze/lapositiva/evaluacion2

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: workspace
      volume_base: /Volumes/workspace/bronze/lapositiva/evaluacion2
```

11.2 Pipeline resource

```
resources:
  pipelines:
    eval2_gold:
      name: PIPELINE_EVAL2_GOLD
      catalog: ${var.catalog}
      schema: gold
      serverless: true
      continuous: false
      libraries:
        - glob:
            include: ../src/03_gold_pipeline_eval2.sql
```

11.3 Validar, desplegar y ejecutar

```
# Ejecutar desde la raíz del repositorio / Bundle

databricks bundle validate -t dev

databricks bundle deploy -t dev

databricks bundle run -t dev eval2_job
```

Resultado esperado. Después de deploy, Databricks debe mostrar PIPELINE_EVAL2_GOLD y JOB_EVAL2_SEGUROS. El Job debe finalizar en SUCCESS.

12 Guardar los cambios y publicar en Bitbucket

Los cambios del código y YAML se suben después de validar el flujo.

12.1 Revisar cambios locales

```
git status
```

12.2 Preparar y confirmar cambios

```
git add .
git commit -m "Evaluacion 2 guiada - volumen, pipeline y job"
```

12.3 Publicar

```
git push -u origin feature/evaluacion2
```

12.4 Pull Request

- Crear Pull Request desde feature/evaluacion2 hacia main.
- Verificar que no existan CSV, secretos ni credenciales en los cambios.
- La evidencia debe mostrar la rama, el commit y los archivos YAML del Job/Pipeline.

13 Validaciones y evidencias

Cerrar la evaluación demostrando datos, DAGs, ejecución y versionamiento.

13.1 SQL Editor

```
-- ============================================================
-- EVALUACIÓN 2 GUIADA - VALIDACIÓN EN SQL EDITOR
-- ============================================================

-- 1. Cantidad final de siniestros Silver.
SELECT COUNT(*) AS cantidad_silver
FROM workspace.silver.eval2_siniestros;

-- 2. Debe devolver cero filas.
SELECT id_siniestro, COUNT(*) AS cantidad
FROM workspace.silver.eval2_siniestros
GROUP BY id_siniestro
HAVING COUNT(*) > 1;

-- 3. Verificar schema evolution.
SELECT
  id_siniestro,
  prioridad,
  estado_siniestro,
  fecha_actualizacion
FROM workspace.silver.eval2_siniestros
WHERE prioridad IS NOT NULL
ORDER BY fecha_actualizacion DESC;

-- 4. Gold.
SELECT *
FROM workspace.gold.eval2_resumen_siniestros
ORDER BY reserva_total DESC;

-- 5. Detalle de siniestros y pagos.
SELECT
  s.id_siniestro,
  s.tipo_siniestro,
  s.estado_siniestro,
  p.id_pago,
  p.estado_pago,
  p.monto_pago
FROM workspace.silver.eval2_siniestros s
LEFT JOIN workspace.silver.eval2_pagos p
  ON s.id_siniestro = p.id_siniestro
ORDER BY s.id_siniestro, p.fecha_pago;
```

13.2 Checklist de evidencias

#	Evidencia
1	Volume workspace.bronze.lapositiva creado como EXTERNAL.
2	3 CSV visibles en /Volumes/workspace/bronze/lapositiva/evaluacion2/.
3	Repositorio Bitbucket + rama feature/evaluacion2.
4	3 tablas Bronze con metadata técnica.
5	Silver final con 40 IDs, prioridad y cero duplicados.
6	Pipeline Graph de Gold.
7	JOB_EVAL2_SEGUROS con DAG y ejecución SUCCESS.

#	Evidencia
8	Resultado Gold en SQL Editor.
9	Commit/push final y Pull Request.
10	YAML del Job y Pipeline visibles en Bitbucket.

14

Rúbrica propuesta

Evaluación guiada: se prioriza ejecución correcta, trazabilidad y comprensión del flujo.

Criterio	Puntos	Evidencia
Volume externo y carga de archivos	10	Volume lapositiva, rutas correctas y archivos visibles.
Bitbucket / Git Folder	10	Repo, credencial, branch y sincronización.
Bronze + metadata	15	Lectura, schemas, Delta y columnas técnicas.
Silver + MERGE	25	Window, limpieza, schema evolution e idempotencia.
Gold Declarative Pipeline	15	2 Materialized Views y DAG correcto.
Lakeflow Job / Bundle	15	YAML versionado, deploy y SUCCESS.
SQL + evidencias finales	10	Consultas de control y trazabilidad.

14.1 Resultados de control docente

Control	Esperado
Bronze siniestros base	48 filas
Bronze incremental	13 filas
Bronze pagos	60 filas
Silver final siniestros	40 IDs únicos
Silver pagos	55 IDs de pago únicos
Duplicados Silver	0
Gold	5 tipos de siniestro

A

Anexo A - Código Bronze

Archivo src/01_bronze_eval2.py

```
# Databricks notebook source
# ============================================================
# EVALUACIÓN 2 GUIADA - 01 BRONZE
# Volume -> Delta Bronze
# ============================================================

# COMMAND ----------
from pyspark.sql import functions as F
from pyspark.sql.types import *
import uuid

# Job parameters. Cuando se ejecuta manualmente, estos widgets
# también funcionan como valores por defecto.
dbutils.widgets.text("catalog", "workspace")
dbutils.widgets.text(
    "volume_base",
    "/Volumes/workspace/bronze/lapositiva/evaluacion2"
)

catalog = dbutils.widgets.get("catalog")
volume_base = dbutils.widgets.get("volume_base")
run_id = str(uuid.uuid4())

base_path = f"{volume_base}/siniestros/siniestros_eval2_base.csv"
inc_path = f"{volume_base}/siniestros/siniestros_eval2_incremental.csv"
pagos_path = f"{volume_base}/pagos/pagos_eval2.csv"

# COMMAND ----------
# Schemas

schema_siniestros_base = StructType([
    StructField("id_siniestro", StringType(), True),
    StructField("id_asegurado", StringType(), True),
    StructField("tipo_siniestro", StringType(), True),
    StructField("fecha_ocurrencia", DateType(), True),
    StructField("fecha_reporte", DateType(), True),
    StructField("estado_siniestro", StringType(), True),
    StructField("monto_reserva", DoubleType(), True),
    StructField("departamento", StringType(), True),
    StructField("canal_reporte", StringType(), True),
    StructField("fecha_actualizacion", TimestampType(), True),
])

schema_siniestros_incremental = StructType(
    schema_siniestros_base.fields + [
        StructField("prioridad", StringType(), True)
    ]
)

schema_pagos = StructType([
    StructField("id_pago", StringType(), True),
    StructField("id_siniestro", StringType(), True),
    StructField("fecha_pago", DateType(), True),
    StructField("monto_pago", DoubleType(), True),
    StructField("estado_pago", StringType(), True),
    StructField("medio_pago", StringType(), True),
])

# COMMAND ----------
def read_csv(path, schema):
    return (
        spark.read
        .option("header", True)
        .option("dateFormat", "yyyy-MM-dd")
        .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
        .schema(schema)
        .csv(path)
    )

def add_metadata(df, filename):
    return (
        df
        .withColumn("fecha_ingestion", F.current_timestamp())
        .withColumn("archivo_origen", F.lit(filename))
        .withColumn("id_ejecucion", F.lit(run_id))
    )

df_base = add_metadata(
    read_csv(base_path, schema_siniestros_base),
    "siniestros_eval2_base.csv"
)

df_inc = add_metadata(
    read_csv(inc_path, schema_siniestros_incremental),
    "siniestros_eval2_incremental.csv"
)

df_pagos = add_metadata(
    read_csv(pagos_path, schema_pagos),
    "pagos_eval2.csv"
)

# COMMAND ----------
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.bronze")

tables = {
    f"{catalog}.bronze.eval2_siniestros_base": df_base,
    f"{catalog}.bronze.eval2_siniestros_incremental": df_inc,
    f"{catalog}.bronze.eval2_pagos": df_pagos,
}

for table_name, df in tables.items():
    (
        df.write
        .format("delta")
        .mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable(table_name)
    )
```

B

Anexo B - Código Silver + MERGE

Archivo src/02_silver_eval2.py

```
# Databricks notebook source
# ============================================================
# EVALUACIÓN 2 GUIADA - 02 SILVER + MERGE
# Window + limpieza + schema evolution + idempotencia
# ============================================================

# COMMAND ----------
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from delta.tables import DeltaTable
import uuid

dbutils.widgets.text("catalog", "workspace")
dbutils.widgets.text(
    "volume_base",
    "/Volumes/workspace/bronze/lapositiva/evaluacion2"
)

catalog = dbutils.widgets.get("catalog")
run_id = str(uuid.uuid4())

BASE = f"{catalog}.bronze.eval2_siniestros_base"
INC = f"{catalog}.bronze.eval2_siniestros_incremental"
PAGOS = f"{catalog}.bronze.eval2_pagos"

SILVER_SIN = f"{catalog}.silver.eval2_siniestros"
SILVER_PAGOS = f"{catalog}.silver.eval2_pagos"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.silver")

# COMMAND ----------
# 1. Última versión del histórico por id_siniestro

w = (
    Window
    .partitionBy("id_siniestro")
    .orderBy(F.col("fecha_actualizacion").desc())
)

base_latest = (
    spark.table(BASE)
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1)
    .drop("rn")
    .withColumn("fecha_procesamiento", F.current_timestamp())
    .withColumn("id_ejecucion_silver", F.lit(run_id))
)

(
    base_latest.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .saveAsTable(SILVER_SIN)
)

print("Silver inicial:", spark.table(SILVER_SIN).count())

# COMMAND ----------
# 2. Deduplicar lote incremental

inc_latest = (
    spark.table(INC)
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1)
    .drop("rn")
    .withColumn("fecha_procesamiento", F.current_timestamp())
    .withColumn("id_ejecucion_silver", F.lit(run_id))
)

# COMMAND ----------
# 3. MERGE / UPSERT con evolución de schema
# La columna prioridad llega solamente en el incremental.

target = DeltaTable.forName(spark, SILVER_SIN)

def ejecutar_merge():
    (
        target.alias("target")
        .merge(
            inc_latest.alias("source"),
            "target.id_siniestro = source.id_siniestro"
        )
        .withSchemaEvolution()
        .whenMatchedUpdateAll(
            condition=(
                "source.fecha_actualizacion > "
                "target.fecha_actualizacion"
            )
        )
        .whenNotMatchedInsertAll()
        .execute()
    )

ejecutar_merge()
count_1 = spark.table(SILVER_SIN).count()

# Segunda ejecución: mismo lote, mismo resultado.
ejecutar_merge()
count_2 = spark.table(SILVER_SIN).count()

print("Después del MERGE 1:", count_1)
print("Después del MERGE 2:", count_2)
print("Idempotencia OK:", count_1 == count_2)

# COMMAND ----------
# 4. Limpieza de pagos

pagos = (
```

C

Anexo C - Declarative Pipeline Gold

Archivo src/03_gold_pipeline_eval2.sql

```
-- ============================================================
-- EVALUACIÓN 2 GUIADA - 03 GOLD DECLARATIVE PIPELINE
-- Se despliega como PIPELINE_EVAL2_GOLD
-- Catalog: workspace
-- Schema: gold
-- ============================================================

CREATE OR REFRESH MATERIALIZED VIEW eval2_pagos_por_siniestro
COMMENT 'Pagos procesados agregados por siniestro'
AS
SELECT
  id_siniestro,
  COUNT(*) AS cantidad_pagos,
  SUM(CASE WHEN estado_pago = 'PROCESADO' THEN 1 ELSE 0 END)
    AS cantidad_pagos_procesados,
  ROUND(
    SUM(
      CASE
        WHEN estado_pago = 'PROCESADO'
        THEN COALESCE(monto_pago, 0)
        ELSE 0
      END
    ),
    2
  ) AS monto_pagado
FROM workspace.silver.eval2_pagos
GROUP BY id_siniestro;


CREATE OR REFRESH MATERIALIZED VIEW eval2_resumen_siniestros
COMMENT 'Resumen analítico de siniestros y pagos'
AS
SELECT
  s.tipo_siniestro,
  COUNT(DISTINCT s.id_siniestro) AS cantidad_siniestros,
  ROUND(SUM(s.monto_reserva), 2) AS reserva_total,
  ROUND(SUM(COALESCE(p.monto_pagado, 0)), 2) AS monto_pagado,
  SUM(COALESCE(p.cantidad_pagos_procesados, 0))
    AS pagos_procesados
FROM workspace.silver.eval2_siniestros s
LEFT JOIN eval2_pagos_por_siniestro p
  ON s.id_siniestro = p.id_siniestro
GROUP BY s.tipo_siniestro;
```

D

Anexo D - Validación del Job

Archivo src/04_validacion_eval2.py

```
# Databricks notebook source
# ============================================================
# EVALUACIÓN 2 GUIADA - 04 VALIDACIÓN
# ============================================================

# COMMAND ----------
dbutils.widgets.text("catalog", "workspace")
catalog = dbutils.widgets.get("catalog")

silver_sin = f"{catalog}.silver.eval2_siniestros"
gold = f"{catalog}.gold.eval2_resumen_siniestros"

count_silver = spark.table(silver_sin).count()

duplicados = (
    spark.table(silver_sin)
    .groupBy("id_siniestro")
    .count()
    .filter("count > 1")
    .count()
)

count_gold = spark.table(gold).count()

print("Silver:", count_silver)
print("Duplicados:", duplicados)
print("Gold:", count_gold)

calidad_ok = (
    "OK"
    if duplicados == 0 and count_gold > 0
    else "ERROR"
)

dbutils.jobs.taskValues.set(
    key="calidad_ok",
    value=calidad_ok
)

print("calidad_ok:", calidad_ok)
display(spark.table(gold))
```

R

Referencias técnicas verificadas

Documentación oficial vigente de Databricks consultada para esta versión de revisión.

Create and manage Unity Catalog volumes: https://docs.databricks.com/aws/en/volumes/utility-commands

Connect your Git provider to Databricks: https://docs.databricks.com/aws/en/repos/get-access-tokens-from-git-provider

Create and manage Git folders: https://docs.databricks.com/aws/en/repos/git-operations-with-repos

What are Declarative Automation Bundles?: https://docs.databricks.com/aws/en/dev-tools/bundles

Develop a job with Declarative Automation Bundles: https://docs.databricks.com/aws/en/dev-tools/bundles/jobs-tutorial

Declarative Automation Bundles resources: https://docs.databricks.com/aws/en/dev-tools/bundles/resources

Nota de revisión. Las pantallas y nombres de menús pueden cambiar ligeramente. La versión final debe reemplazar <S3_URI_LAPOSITIVA> y <BITBUCKET_REPO_URL> con los valores reales del laboratorio.